

UNIT SECOND :-Exceptions Object Oriented design & Functional Programming

Errors and Exceptions in Python

Errors are problems in a program that causes the program to stop its execution. On the other hand, exceptions are raised when some internal events change the program's normal flow.

Syntax Errors in Python

Syntax error occurs when the code doesn't follow Python's rules, like using incorrect grammar in English. Python stops and points out the issue before running the program.

Example 1 : In this example, this code returns a syntax error because there is a missing colon (:) after the if statement. The correct syntax requires a colon to indicate the start of the block of code to be executed if the condition is true.

```
a = 10000
```

```
if a > 2999
```

```
    print("Eligible")
```

Output

```
File "<ipython-input-1-94302ab5dc97>", line 3
    if a > 2999
        ^
SyntaxError: expected ':'
```

Example 2: This code returns a syntax error because parentheses around the condition in an if statement are not required in Python. While optional, their unnecessary use can lead to incorrect syntax.

```
if a<3:
```

```
    print("gfg")
```

Output

```
File "<ipython-input-2-9d7cace61905>", line 2
    print("gfg")
    ^
IndentationError: expected an indented block after 'if' statement on line
```

Python Logical Errors (Exceptions)

Logical errors are subtle bugs in the program that allow the code to run, but produce incorrect or unintended results. These are often harder to detect since the program doesn't crash, but the output is not as expected.

Characteristics of Logical Errors

- **No Syntax Error:** The code runs without issues.
- **Unexpected Output:** The results produced by the program are not what the programmer intended.
- **Difficult to Detect:** These errors can be tricky to spot since there's no obvious problem with the code itself.
- **Causes:** Faulty logic, incorrect assumptions, or improper use of operators.

Example:

```
a = [10, 20, 30, 40, 50]
```

```
b = 0
```

```
for i in a:
```

```
    b += i
```

```
res = b / len(a) - 1
```

```
print(res)
```

Output

```
29.0
```

Explanation: Expected output is that the average of a should be 30, but the program outputs 29.0. The logical error occurs because the formula **b / len(a) - 1** incorrectly subtracts 1, leading to an incorrect result. The correct formula should be **b / len(a)**.

Error Handling

Python provides mechanisms to handle errors and exceptions using the try, except, and finally blocks. This allows for graceful handling of errors without crashing the program.

Example 1: This example shows how try, except and finally handle errors. try runs risky code, except catches errors and finally runs always.

```
try:
```

```
print("code start")

print(1 / 0)

except:

    print("an error occurs")

finally:

    print("GeeksForGeeks")
```

Output

```
code start
an error occurs
GeeksForGeeks
```

Explanation:

- **try block** runs code, raising an exception if any occurs (e.g., 1 / 0 raises ZeroDivisionError).
- **except block** catches exceptions, printing "an error occurs" in case of an error.
- **finally block** always executes, printing "GeeksForGeeks" to signal the end of execution.

Python Exception Handling

Python Exception Handling allows a program to gracefully handle unexpected events (like invalid input or missing files) without crashing. Instead of terminating abruptly, Python lets you detect the problem, respond to it, and continue execution when possible.

Let's see an example to understand it better:

Basic Example: Handling Simple Exception

Here's a basic example demonstrating how to catch an exception and handle it gracefully:

```
n = 10
```

```
try:
```

```
    res = n / 0
```

```
except ZeroDivisionError:
```

```
    print("Can't be divided by zero!")
```

Output

```
Can't be divided by zero!
```

Explanation: Dividing a number by 0 raises a [ZeroDivisionError](#). The try block contains code that may fail and except block catches the error, printing a safe message instead of stopping the program.

Difference Between Errors and Exceptions

Errors and exceptions are both issues in a program, but they differ in severity and handling. Let's see how:

- **Error:** Serious problems in the program logic that cannot be handled. Examples include syntax errors or memory errors.
- **Exception:** Less severe problems that occur at runtime and can be managed using exception handling (e.g., invalid input, missing files).

Example: This example shows the difference between a syntax error and a runtime exception

Syntax Error (Error)

```
print("Hello world" # Missing closing parenthesis
```

ZeroDivisionError (Exception)

```
n = 10
```

```
res = n / 0
```

Explanation: A syntax error stops the code from running at all, while an exception like ZeroDivisionError occurs during execution and can be caught with exception handling.

Python Classes and Objects

Python's object-oriented programming (OOP) allows you to model real-world entities in code, making programs more organized, reusable and easier to maintain. By grouping related data and behavior into a single unit, classes and objects help you write cleaner, more logical code for everything from small scripts to large applications.

Class

A class in Python is a user-defined template for creating objects. It bundles data and functions together, making it easier to manage and use them. When we create a new class, we define a new type of object. We can then create multiple instances of this object type.

**Creating Class**

Classes are created using **class keyword**. Attributes are variables defined inside class and represent properties of the class. Attributes can be accessed using dot **.** **operator** (e.g., MyClass.my_attribute).

define a class

```
class Dog:
```

```
    sound = "bark" # class attribute
```

Object

An **object** is a specific instance of a class. It holds its own set of data (instance variables) and can invoke methods defined by its class. Multiple objects can be created from same class, each with its own unique attributes. Let's create an object from **Dog class**.

```
class Dog:
```

```
    sound = "bark"
```

```
dog1 = Dog() # Creating object from class
```

```
print(dog1.sound) # Accessing the class
```

Output

```
bark
```

Explanation: **sound attribute** is a class attribute. It is shared across all instances of Dog class, so can be directly accessed through instance **dog1**.

Using `__init__()` Function

In Python, class has `__init__()` function which automatically initializes object attributes when an object is created. The `__init__()` method is the constructor in Python.

Let's see how to create a class with `__init__()` to set up attributes when an object is made:

```
class Dog:
```

```
    species = "Canine" # Class attribute
```

```
    def __init__(self, name, age):
```

```
self.name = name # Instance attribute
```

```
self.age = age # Instance attribute
```

Explanation:

- **class Dog:** Defines a class named Dog.
- **species:** class attribute shared by all instances of class.
- **__init__ method:** Initializes name and age attributes when a new object is created.

Initiate Object with __init__()

Let's now create an object using `__init__()` method and see how it initializes attributes:

```
class Dog:
```

```
    species = "Canine" # Class attribute
```

```
    def __init__(self, name, age):
```

```
        self.name = name # Instance attribute
```

```
        self.age = age # Instance attribute
```

```
# Creating an object of the Dog class
```

```
dog1 = Dog("Buddy", 3)
```

```
print(dog1.name)
```

```
print(dog1.species)
```

Output

```
Buddy
```

```
Canine
```

Explanation:

- **Dog("Buddy", 3):** Creates an object of Dog class with name as "Buddy" and age as 3.
- **dog1.name:** Accesses instance attribute name of dog1 object.
- **dog1.species:** Accesses class attribute species of dog1 object.

Self Parameter

self parameter is a reference to the current instance of class. It allows us to access the attributes and methods of the object.

Let's see an example of how the self parameter works in accessing attributes and methods:

```
class Dog:

    def __init__(self, name, age):

        self.name = name

        self.age = age

    def bark(self):

        print(f'{self.name} is barking!')
```

Creating an instance of Dog

```
dog1 = Dog("Buddy", 3)
```

```
dog1.bark()
```

Output

```
Buddy is barking!
```

Explanation:

- Inside **bark()**, **self.name** accesses specific dog's name and prints it.
- When we call **dog1.bark()**, [Python](#) automatically passes dog1 as self, allowing access to its attributes.

Instance Variables

Variables that are unique to each instance (object) of a class. These are defined within **__init__()** method or other instance methods. Each object maintains its own copy of instance variables, independent of other objects.

Example:

This code shows how class variables are shared across all objects, while instance variables are unique to each object.

```
class Dog:

    # Class variable

    species = "Canine"

    def __init__(self, name, age):
```

```
# Instance variables

self.name = name

self.age = age

# Create objects

dog1 = Dog("Buddy", 3)

dog2 = Dog("Charlie", 5)


# Access class and instance variables

print(dog1.species) # (Class variable)

print(dog1.name)    # (Instance variable)

print(dog2.name)    # (Instance variable)

# Modify instance variables

dog1.name = "Max"

print(dog1.name)    # (Updated instance variable)

# Modify class variable

Dog.species = "Feline"

print(dog1.species) # (Updated class variable)

print(dog2.species)
```

Explanation:

- **Class Variable (species):** Shared by all instances of class. Changing Dog.species affects all objects, as it's a property of class itself.
- **Instance Variables (name, age):** Defined in __init__() method. Unique to each instance (e.g., dog1.name and dog2.name are different).

- **Accessing Variables:** Class variables can be accessed via class name (Dog.species) or an object (dog1.species). Instance variables are accessed via object (dog1.name).
- **Updating Variables:** Changing Dog.species affects all instances. Changing dog1.name only affects dog1 and does not impact dog2.

Constructors in Python

In Python, a constructor is a special method that is called automatically when an object is created from a class. Its main role is to initialize the object by setting up its attributes or state.

The method `__new__` is the **constructor** that creates a new instance of the class while `__init__` is the initializer that sets up the instance's attributes after creation. These methods work together to manage object creation and initialization.

`__new__` Method

This method is responsible for **creating a new instance of a class**. It allocates memory and **returns the new object**. It is called before `__init__`.

```
class ClassName:
```

```
    def __new__(cls, parameters):
```

```
        instance = super(ClassName, cls).__new__(cls)
```

```
        return instance
```

To learn more, please refer to "`__new__`" method

`__init__` Method

This method initializes the newly created instance and is commonly used as a constructor in Python. It is called immediately after the object is created by `__new__` method and is responsible for initializing attributes of the instance.

Syntax:

```
class ClassName:
```

```
    def __init__(self, parameters):
```

```
        self.attribute = value
```

Note: It is called after `__new__` and does not return anything (it returns **None** by default).

To learn more, please refer to "`__init__`" method

Types of Constructors

Constructors can be of two types.

1. Default Constructor

A **default constructor** does not take any parameters other than **self**. It initializes the object with default attribute values.

```
class Car:
```

```
    def __init__(self):
```

```
        #Initialize the Car with default attributes
```

```
        self.make = "Toyota"
```

```
        self.model = "Corolla"
```

```
        self.year = 2020
```

```
# Creating an instance using the default constructor
```

```
car = Car()
```

```
print(car.make)
```

```
print(car.model)
```

```
print(car.year)
```

Output

```
Toyota
```

```
Corolla
```

```
2020
```

2. Parameterized Constructor

A **parameterized constructor** accepts arguments to initialize the object's attributes with specific values.

```
class Car:
```

```
    def __init__(self, make, model, year):
```

```
#Initialize the Car with specific attributes.  
  
self.make = make  
  
self.model = model  
  
self.year = year  
  
# Creating an instance using the parameterized constructor  
  
car = Car("Honda", "Civic", 2022)  
  
print(car.make)  
  
print(car.model)  
  
print(car.year)
```

Output

```
Honda  
Civic  
2022
```

Classes With Multiple Objects:-

What is a Class with Multiple Objects?

- A **class** is a blueprint for creating objects.
- You can create **many objects** from the same class, and each object can store its own data.

Example: Student Class with Multiple Objects

class Student:

```
def __init__(self, name, age, grade):
```

```
    self.name = name
```

```
    self.age = age
```

```
    self.grade = grade
```

```
def display(self):
```

```
    print(f"Name: {self.name}, Age: {self.age}, Grade: {self.grade}")
```

```
# Creating multiple objects
```

```
s1 = Student("Amit", 16, "10th")
```

```
s2 = Student("Priya", 15, "9th")
```

```
s3 = Student("Rahul", 17, "11th")
```

```
# Displaying details of each object
```

```
s1.display()
```

```
s2.display()
```

```
s3.display()
```

Output:

```
Name: Amit, Age: 16, Grade: 10th  
Name: Priya, Age: 15, Grade: 9th  
Name: Rahul, Age: 17, Grade: 11th
```

Explanation:

- Student is the **class** (blueprint).
- s1, s2, s3 are **different objects** (instances).
- Each object has its **own values** for name, age, and grade.
- The same method `display()` works for all objects.

Encapsulation:-

Encapsulation means hiding internal details of a class and only exposing what's necessary. It helps to protect important data from being changed directly and keeps the code secure and organized.

This example shows encapsulation by keeping `__salary` variable private inside Employee class. It cannot be accessed directly from outside the class.

class Employee:

```
    def __init__(self, name, salary):
```

```
        self.name = name        # public attribute
```

```
self.__salary = salary # private attribute

emp = Employee("Fedrick", 50000)

print(emp.name)

print(emp.__salary)
```

Output

Fedrick

ERROR!

Traceback (most recent call last):

File "<main.py>", line 8, in <module>

AttributeError: 'Employee' object has no attribute '__salary'

Explanation:

- **self.name = name:** Public attribute, can be accessed directly.
- **self.__salary = salary:** Private attribute, cannot be accessed directly.
- **print(emp.name):** Prints "Fedrick" because name is public.
- **print(emp.__salary):** Raises an error because __salary is private and hidden.

Why do we need Encapsulation?

- Protects data from unauthorized access and accidental modification.
- Controls data updates using getter/setter methods with validation.
- Enhances modularity by hiding internal implementation details.
- Simplifies maintenance through centralized data handling logic.
- Reflects real-world scenarios like restricting direct access to a bank account balance.

Inheritance:-

Inheritance is a fundamental concept in object-oriented programming (OOP) that allows a class (called a child or derived class) to inherit attributes and methods from another class (called a parent or base class). In this article, we'll explore inheritance in Python.

Example: Here, we create a parent class Animal that has a method info(). Then we create a child classes Dog that inherit from Animal and add their own behavior.

class Animal:

```
def __init__(self, name):
```

```
    self.name = name
```

```
def info(self):
```

```
    print("Animal name:", self.name)
```

class Dog(Animal):

```
def sound(self):
```

```
    print(self.name, "barks")
```

```
d = Dog("Buddy")
```

```
d.info()    # Inherited method
```

```
d.sound()
```

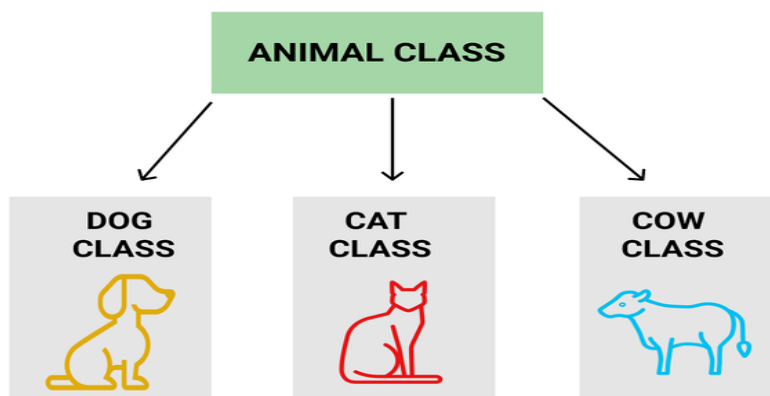
Output

Animal name: Buddy

Buddy barks

Explanation:

- **class Animal:** Defines the parent class.
- **info():** Prints the name of the animal.
- **class Dog(Animal):** Defines Dog as a child of Animal class.
- **d.info():** Calls parent method info() and **d.sound():** Calls child method.



Types of Python Inheritance

1. Single Inheritance

In single inheritance, a child class inherits from just one parent class.

Example: This example shows a child class Employee inheriting a property from the parent class Person.

```
class Person:
```

```
    def __init__(self, name):
```

```
        self.name = name
```

```
class Employee(Person): # Employee inherits from Person
```

```
def show_role(self):  
    print(self.name, "is an employee")  
  
emp = Employee("Sarah")  
  
print("Name:", emp.name)  
  
emp.show_role()
```

Output

Name: Sarah

Sarah is an employee

2. Multiple Inheritance

In multiple inheritance, a child class can inherit from more than one parent class.

Example: This example demonstrates Employee inheriting properties from two parent classes: Person and Job.

class Person:

```
    def __init__(self, name):  
        self.name = name
```

class Job:

```
    def __init__(self, salary):  
        self.salary = salary
```

class Employee(Person, Job): # Inherits from both Person and Job

```
    def __init__(self, name, salary):  
        Person.__init__(self, name)  
        Job.__init__(self, salary)  
  
    def details(self):  
        print(self.name, "earns", self.salary)
```

```
emp = Employee("Jennifer", 50000)  
  
emp.details()
```

Output

Jennifer earns 50000

Explanation: Here Employee gets attributes from both Person and Job and It can access both name and salary.

3. Multilevel Inheritance

In multilevel inheritance, a class is derived from another derived class (like a chain).

Example: This example shows Manager inheriting from Employee, which in turn inherits from Person.

```
class Person:
```

```
    def __init__(self, name):  
        self.name = name
```

```
class Employee(Person):
```

```
    def show_role(self):  
        print(self.name, "is an employee")
```

```
class Manager(Employee): # Manager inherits from Employee
```

```
    def department(self, dept):  
        print(self.name, "manages", dept, "department")
```

```
mgr = Manager("Joy")
```

```
mgr.show_role()
```

```
mgr.department("HR")
```

Output

```
Joy is an employee
```

```
Joy manages HR department
```

Explanation: Here Manager inherits from Employee and Employee inherits from Person. So Manager can use methods from both parent and grandparent.

4. Hierarchical Inheritance

In hierarchical inheritance, multiple child classes inherit from the same parent class.

Example: This example demonstrates two child classes (Employee and Intern) inheriting from a single parent class Person.

```
class Person:
```

```
    def __init__(self, name):  
        self.name = name
```

```
class Employee(Person):
```

```
    def role(self):  
        print(self.name, "works as an employee")
```

```
class Intern(Person):
```

```
    def role(self):  
        print(self.name, "is an intern")
```

```
emp = Employee("David")
```

```
emp.role()
```

```
intern = Intern("Eva")  
intern.role()
```

Output

```
David works as an employee
```

```
Eva is an intern
```

Explanation: Both Employee and Intern inherit from Person. They share the parent's property (name) but implement their own methods.

5. Hybrid Inheritance

Hybrid inheritance is a combination of more than one type of inheritance.

Example: This example demonstrates TeamLead inheriting from both Employee (which inherits Person) and Project, combining multiple inheritance types.

```
class Person:  
  
    def __init__(self, name):  
        self.name = name  
  
class Employee(Person):  
  
    def role(self):  
        print(self.name, "is an employee")  
  
class Project:  
  
    def __init__(self, project_name):  
        self.project_name = project_name  
  
class TeamLead(Employee, Project): # Hybrid Inheritance  
  
    def __init__(self, name, project_name):  
        Employee.__init__(self, name)  
        Project.__init__(self, project_name)  
  
    def details(self):  
        print(self.name, "leads project:", self.project_name)  
  
lead = TeamLead("Sophia", "AI Development")  
  
lead.role()
```

```
lead.details()
```

Output

```
Sophia is an employee
```

```
Sophia leads project: AI Development
```

Explanation: Here TeamLead inherits from Employee (which already inherits Person) and also from Project. This combines single, multilevel and multiple inheritance -> hybrid.

Polymorphism:-

Polymorphism means "many forms". It refers to the ability of an entity (like a function or object) to perform different actions based on the context.

Technically, in Python, polymorphism allows same method, function or operator to behave differently depending on object it is working with. This makes code more flexible and reusable.

Why do we need Polymorphism?

- Ensures consistent interfaces across different classes.
- Allows objects to respond differently to the same method call.
- Promotes loose coupling by relying on shared behavior, not specific types.
- Enables writing flexible, reusable code that works across types.
- Simplifies testing and future extension of code.

Example of Polymorphism

A remote control can operate multiple devices like a TV, AC or music system. You press the power button and each device responds differently TV turns on, AC starts cooling, music system plays music.

Polymorphism here means same interface (power button), but different behavior based on device (object).

Types of Polymorphism

Polymorphism in Python refers to ability of the same method or operation to behave differently based on object or context. It mainly includes **compile-time** and **runtime polymorphism**.

Let's discuss each type in detail.

1. Compile-time Polymorphism

Compile-time polymorphism means deciding which method or operation to run during compilation, usually through method or operator overloading.

Languages like Java or C++ support this. But Python doesn't because it's dynamically typed it resolves method calls at runtime, not during compilation. So, true method overloading isn't supported in Python, though similar behavior can be achieved using default or variable arguments.

Example:

This code demonstrates method overloading in Python using default and variable-length arguments. The multiply() method works with different numbers of inputs, mimicking compile-time polymorphism

class Calculator:

```
def multiply(self, a=1, b=1, *args):  
  
    result = a * b  
  
    for num in args:
```

```
        result *= num

    return result

# Create object

calc = Calculator()

# Using default arguments

print(calc.multiply())

print(calc.multiply(4))

# Using multiple arguments

print(calc.multiply(2, 3))

print(calc.multiply(2, 3, 4))
```

Output

```
1
4
6
24
```

2. Runtime Polymorphism (Overriding)

Runtime polymorphism means that the behavior of a method is decided while program is running, based on the object calling it.

In Python, this happens through **Method Overriding** a child class provides its own version of a method already defined in the parent class. Since Python is dynamic, it supports this, allowing same method call to behave differently for different object types.

Example:

This code shows runtime polymorphism using method overriding. The sound() method is defined in base class Animal and overridden in Dog and Cat. At runtime, correct method is called based on object's class.

```
class Animal:
```

```
    def sound(self):
```

```
        return "Some generic sound"
```

```
class Dog(Animal):
```

```
    def sound(self):
```

```
        return "Bark"

class Cat(Animal):

    def sound(self):

        return "Meow"

# Polymorphic behavior

animals = [Dog(), Cat(), Animal()]

for animal in animals:

    print(animal.sound())
```

Output

```
Bark
Meow
Some generic sound
```

Explanation: Here, sound method behaves differently depending on whether object is a Dog, Cat or Animal and this decision happens at runtime. This dynamic nature makes Python particularly powerful for runtime polymorphism.

Lambda :-

In Python, a lambda function is an anonymous function meaning it is defined without a name. Unlike regular functions defined using def keyword, lambda functions are created using lambda keyword

They are useful for writing small, simple functions in a concise way, especially when you need a function temporarily.

Syntax

lambda arguments : expression

- **arguments:** Input parameters (can be zero or more).
- **expression:** A single expression evaluated and returned by the function.

Basic Example

Here's a simple lambda function that checks if a number is even or odd:

```
calc = lambda num: "Even number" if num % 2 == 0 else "Odd number"

print(calc(20))
```

Output

```
Even number
```

Explanation: calc creates an anonymous function that checks if a number is divisible by 2. If yes, returns "Even number", otherwise "Odd number".

Key Properties of Lambda Functions

1. Lambda functions can have any number of arguments but only one expression.
2. The expression is evaluated and returned automatically no explicit **return** needed.
3. Lambda functions are syntactically restricted to a **single expression** (no multiple statements or commands).
4. They are often used where small, one-off function objects are required for example, as arguments to other functions.

Examples of Lambda Function**Example 1: Lambda Returns a Function Object**

This code prints the lambda function object itself instead of executing it.

```
string = 'GeeksforGeeks'
```

```
print(lambda string: string) # Lambda here returns a function object, not a value
```

Output

```
<function <lambda> at 0x7f42b0b7f100>
```

Explanation:

- **lambda string: string** - Defines an anonymous function that takes one argument (string) and returns it unchanged.
- **print(lambda string: string)** - Prints the function object itself, not its output, because lambda is not called here.
- Output **<function <lambda> at 0x7fd7517ade18>** is Python's way of showing memory address where the function is stored.

Iterator

An iterator in Python is an object used to traverse through all the elements of a collection (like lists, tuples or dictionaries) one element at a time. It follows the iterator protocol, which involves two key methods:

- **__iter__()**: Returns the iterator object itself.
- **__next__()**: Returns the next value from the sequence. Raises StopIteration when the sequence ends.

Built-in Iterator Example

Let's start with a simple example using a string. We will convert it into an iterator and fetch characters one by one:

```
s = "GFG"

it = iter(s)

print(next(it))

print(next(it))

print(next(it))
```

Output

```
G
```

F

G

Explanation:

- **s** is an iterable (string).
- **iter(s)** creates an iterator.
- **next(it)** retrieves characters one by one.

Generators in Python

A **generator function** is a special type of function that returns an iterator object. Instead of using return to send back a single value, generator functions use yield to produce a series of results over time. This allows the function to generate values and pause its execution after each yield, maintaining its state between iterations.

```
def fun(max):
```

```
    cnt = 1
```

```
    while cnt <= max:
```

```
        yield cnt
```

```
        cnt += 1
```

```
ctr = fun(5)
```

```
for n in ctr:
```

```
    print(n)
```

Output

1

2

3

4

5

Explanation: This generator function fun yields numbers from 1 up to a specified max. Each call to **next()** on the generator object resumes execution right after the yield statement, where it last left off.

List Comprehension in Python

List comprehension is a concise and powerful way to create new lists by applying an expression to each item in an existing iterable (like a list, tuple or range). It helps you write clean, readable and efficient code compared to traditional loops.

Suppose you want to square every number in a list:

```
a = [2,3,4,5]
```

```
res = [val ** 2 for val in a]
```

```
print(res)
```

Output

```
[4, 9, 16, 25]
```

Explanation: `res = [val ** 2 for val in a]` use list comprehension to create a new list by squaring each number in `a`.

Why Use List Comprehension?

- **Cleaner code:** Combines looping, filtering and transformation in one line.
 - **More readable:** Avoids verbose loops and temporary variables.
 - **Faster execution:** Often faster than equivalent for-loops.
-